The Magic of Bytes Writeup

"I heard my friend wants to prepare for the new AP Cybersecurity CollegeBoard is adding soon, so I prepared a challenge with two python reverse engineering techniques. He insisted on it having an ELF binary though, so I also included it. Can you solve the challenge?"

Solution

We are presented with two files: bytes.txt and chall.py

In chall.py we see that the program shifts the bytes of an ELF file by an unknown constant and outputs the result to bytes.txt

Either by knowing that the magic number of an ELF file is 7F454C46 or by deducing that the large amount of 9's would come from the large amount of 0's in the file, we can determine the shift offset to be 9

Thus, by subtracting the ascii value of each character in bytes.txt by 9, we can get the ELF binary bytes

```
ciphertext = "INSERT BYTES HERE"
def yes_so_fast(ELF_bytes,key):
    message = ""
    for i in range(len(ELF_bytes)):
        message += chr(ord(ELF_bytes[i]) - key)
    print(len(message))
    return message

print(yes_so_fast(ciphertext,9))
```

Inserting the bytes into a hex editor, we can download and run the binary file to reach the next step of the challenge

```
Alright, here's your ELF that you wanted so badly
Now use these bytes for the other python reverse engineering I mentioned
550D0D0A 00000000 41322968 48010000 E3000000 00000000 00000000 00000000 00030000 00400000 00738E00 00006400 64016C00 5A006402 64038400 5A016404 64058400 5A026406 64078400 5A036408 6
4098400 5A04640A 640B8400 5A05640C 640D8400 5A06640E 640F8400 5A076410 64118400 5A086412 64138400 5A09650A 65068300 65038300 17006502 83001700 65048300 17006501 83001700 65058300 17
006508 83001700 65098300 17006507 83001700 83010100 64015300 2914E900 0000004E 63000000 00000000 00000000 00000000 00010000 00430000 00730400 00006401 53002902 4E5A0335 5F49A900 720
20000 00720200 00007202 000000FA 0677696E 2E7079DA 02733103 00000073 02000000 00017204 00000063 00000000 00000000 00000000 00000000 01000000 43000000 73040000 00640153 0029024E 5A03
3331 31720200 00007202 00000072 02000000 72020000 00720300 0000DA02 73320600 00007302 00000000 01720500 00006300 00000000 00000000 00000000 00000000 00000001 00000043 00000073 04000000 64015
300 29024E7A 027B5772 02000000 72020000 00720200 00007202 00000072 03000000 DA027333 09000000 73020000 00000172 06000000 63000000 00000000 00000000 00000000 00010000 00430000 007304
00 00006401 53002902 4E5A045F 54683172 02000000 72020000 00720200 00007202 00000072 03000000 DA027334 0C000000 73020000 00000172 07000000 63000000 00000000 00000000 00000000 0001000
0 00430000 00730400 00006401 53002902 4E5A0435 5F416E72 02000000 72020000 00720200 00007202 00000072 03000000 DA027335 0F000000 73020000 00000172 08000000 63000000 00000000 00000000
 00000000 00010000 00430000 00730400 00006401 53002902 4E5A0553 56424752 72020000 00720200 00007202 00000072 02000000 72030000 00DA0273 36120000 00730200 00000001 72090000 00630000
00000000 00000000 00000000 00000100 00004300 00007304 00000064 01530029 024E7A03 31317D72 02000000 72020000 00720200 00007202 00000072 03000000 DA027337 15000000 73020000 00000172 0
A000000 63000000 00000000 00000000 00000000 00010000 00430000 00730400 00006401 53002902 4E5A055F 354C465F 72020000 00720200 00007202 00000072 02000000 72030000 00DA0273 38180000 00
730200 00000001 720B0000 00630000 00000000 00000000 00000000 00000100 00004300 00007304 00000064 01530029 024E5A03 43484172 02000000 72020000 00720200 00007202 00000072 03000000 DA0
27339 1B000000 73020000 00000172 0C000000 290B5A0A 70795F63 6F6D7069 6C657204 00000072 05000000 72060000 00720700 00007208 00000072 09000000 720A0000 00720B00 0000720C 000000DA 0570
7269 6E747202 00000072 02000000 72020000 00720300 0000DA08 3C6D6F64 756C653E 01000000 73140000 00
```

Once again inserting the bytes into a hex editor, we can either notice that the file is python 3.8 byte-compiled or that the hex data contains "py_compile"

```
550D0D0A 00000000 41322968 48010000 E3000000 00000000 00000000 00000000 00030000 00400000 00738E00 00006400 64016C00 5A006402 64038400 5A016404 64058400
5A026406 64078400 5A036408 64098400 5A04640A 640B8400 5A05640C 640D8400 5A06640E 640F8400 5A076410 64118400 5A086412 64138400 5A09650A 65068300 65038300
17006502 83001700 65048300 17006501 83001700 65058300 17006508 83001700 65098300 17006507 83001700 83010100 64015300 2914E900 0000004E 63000000 00000000
00000000 00000000 00010000 00430000 00730400 00006401 53002902 4E5A0335 5F49A900 72020000 00720200 00007202 000000FA 0677696E 2E7079DA 02733103 00000073
02000000 00017204 00000063 00000000 00000000 00000000 00000000 01000000 43000000 73040000 00640153 0029024E 5A033331 31720200 00007202 00000072 02000000
72020000 00720300 0000DA02 73320600 00007302 00000000 01720500 00006300 00000000 00000000 00000000 00000001 00000043 00000073 04000000 64015300 29024E7A
027B5772 02000000 72020000 00720200 00007202 00000072 03000000 DA027333 09000000 73020000 00000172 06000000 63000000 00000000 00000000 00000000 00010000
00430000 00730400 00006401 53002902 4E5A045F 54683172 02000000 72020000 00720200 00007202 00000072 03000000 DA027334 0C000000 73020000 00000172 07000000
63000000 00000000 00000000 00000000 00010000 00430000 00730400 00006401 53002902 4E5A0435 5F416E72 02000000 72020000 00720200 00007202 00000072 03000000
DA027335 0F000000 73020000 00000172 08000000 63000000 00000000 00000000 00000000 00010000 00430000 00730400 00006401 53002902 4E5A0553 56424752 72020000
00720200 00007202 00000072 02000000 72030000 00DA0273 36120000 00730200 00000001 72090000 00630000 00000000 00000000 00000000 00000100 00004300 00007304
00000064 01530029 024E7A03 31317D72 02000000 72020000 00720200 00007202 00000072 03000000 DA027337 15000000 73020000 00000172 0A000000 63000000 00000000
00000000 00000000 00010000 00430000 00730400 00006401 53002902 4E5A055F 354C465F 72020000 00720200 00007202 00000072 02000000 72030000 00DA0273 38180000
00730200 00000001 720B0000 00630000 00000000 00000000 00000000 00000100 00004300 00007304 00000064 01530029 024E5A03 43484172 02000000 72020000 00720200
00007202 00000072 03000000 DA027339 1B000000 73020000 00000172 0C000000 290B5A0A 70795F63 6F6D7069 6C657204 00000072 05000000 72060000 00720700 00007208
00000072 09000000 720A0000 00720B00 0000720C 000000DA 05707269 6E747202 00000072 02000000 72020000 00720300 0000DA08 3C6D6F64 756C653E 01000000 73140000
00
```

```
U       A2)hH   .                   @   s.   d d l Z d d . Z d d .
Z d d . Z d d . Z d d . Z d d . Z d d . Z d d . Z e e . e .
  e .   e .   e .   e .   e .   e .   e .   .   d S ) .    Nc
              C   s    d S ) NZ 5_I. r    r    r    . win.py. s1    s
      r    c                   C   s    d S ) NZ 311r    r    r
r    r    . s2    s       r    c                   C   s    d S ) Nz
 {Wr    r    r    r    r    . s3    s       r    c
 C   s    d S ) NZ _Th1r    r    r    r    r    . s4    s       r
c                   C   s    d S ) NZ 5_Anr    r    r    r    r
. s5    s       r    c                   C   s    d S ) NZ SVBGRr
 r    r    r    r    . s6    s       r    c                   C   s
   d S ) Nz 11}r    r    r    r    r    . s7    s       r    c
              C   s    d S ) NZ _5LF_r    r    r    r    r    . s8
 s       r    c                   C   s    d S ) NZ CHAr    r    r
  r    r    . s9    s       r    ) Z py_compiler    r    r    r    r
   r    r    r    r    . printr    r    r    r    . <module>    s
```

Either way, we can conclude that we are given python bytecode, which we can decode using uncompyle6 since the python is version 3.8.

By running the command on the bytecode as a .pyc file, we get the decompiled python code that when run gives us the flag.

```
# uncompyle6 version 3.9.2
# Python bytecode version base 3.8.0 (3413)
# Decompiled from: Python 3.12.9 | packaged by Anaconda, Inc. | (main, Feb  6 2025, 12:55:12) [Clang 14.0.6 ]
# Embedded file name: win.py
# Compiled at: 2025-05-17 21:05:05
# Size of source mod 2**32: 328 bytes
import py_compile
```

```
import py_compile

def s1():
    return "5_I"


def s2():
    return "311"


def s3():
    return "{W"


def s4():
    return "_Th1"


def s5():
    return "5 An"


def s6():
    return "SVBGR"


def s7():
    return "11}"


def s8():
    return "_5LF_"


def s9():
    return "CHA"


print(s6() + s3() + s2() + s4() + s1() + s5() + s8() + s9() + s7())
```

```
print(s6() + s3() + s2() + s4() + s1() + s5() + s8() + s9() + s7())
```

```
    ....
    ...: print(s6() + s3() + s2() + s4() + s1() + s5() + s8() + s9() + s7())
SVBGR{W311_Th15_I5_An_5LF_CHA11}
```

`SVBGR{W311_Th15_I5_An_5LF_CHA11}`